

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 2

Important Note: Please download Isabelle from the link provided in the slides and bring a laptop to the large group tutorial. Performing the proofs in Isabelle will be an integral part of those tutorials.

Before you start, please open a new Isabelle/HOL theory file called `Week_1.thy` and add the following to its beginning.

```
theory Week_2
imports Main
begin
```

Exercise 2.1 Addition is Associative

Consider the function $(+)$ defined in Isabelle/HOL for natural numbers.

Prove the following theorem, first pen-and-paper, and then formally in Isabelle.

lemma *add_assoc*: $((a::nat) + b) + c = a + (b + c)$

Hints: 1. Perform the proof by induction, using substitutions *subst*, backward reasoning using the method *rule*, and forward reasoning using *OF*.

2. Think about which variable to perform induction on. Addition on natural numbers is defined by recursion on its first argument: look at *thm add_0_left add_Suc*.

3. You will need to identify multiple lemmas about how *add* works. Do not prove those lemmas. Use *sorry* as the proof. This is the top-down approach.

4. There is a proof which uses only the following lemmas: *add_0_left*, *add_Suc*

Exercise 2.2 Multiplication is Monotone

Consider the multiplication function $(*)$ defined in Isabelle. Prove the following property of it, i.e. that it is monotonically increasing in both arguments. You should prove it both, pen-and-paper and in Isabelle.

lemma *mult_mono*:

fixes *a b c d*:: *nat* — Note the alternative way of fixing the types of the constants

assumes $a \leq b$ $c \leq d$

shows $a * c \leq b * d$

Your pen-and-paper proof should indicate

1. what lemmas are you assuming,
2. on what variable are you performing induction,
3. what are assumptions and the proof goals in the base case and the step case, and what is the induction hypothesis, and
4. how are you instantiating the induction hypothesis, i.e. how do you show that its assumptions hold and which quantified variables are instantiated with which constants.

Hints: 1. The proof should be by induction

2. The following lemmas suffice to prove the goal with only substitution, forward reasoning, and induction:

refl, *diff_le_self*, *mult_le_mono2*, *mult_zero_right*, *le0*, *mult_Suc_right*, *add_mono*, *Suc_eq_plus1*, *le_diff_conv*

3. When you have a term like $x - y$, where x is a *nat*, in the goal or the assumptions, you should perform proof by case analysis on whether $y \leq x$. This is because of the way $-$ is defined for natural numbers, where $x - y = 0$, for any $x \leq y$.

Exercise 2.3 Induction Hypothesis Generalisation

Consider the following function, which adds two natural numbers using an accumulator: in each recursive call it moves one *Suc* from its first argument to its second.

```
fun add_acc :: nat  $\Rightarrow$  nat  $\Rightarrow$  nat where
  add_acc 0 m = m
| add_acc (Suc n) m = add_acc n (Suc m)
```

We want to prove that *add_acc* computes addition.

lemma *add_acc_correct*: *add_acc n m = n + m*

(a) First try to prove the lemma by plain induction on n , i.e. with *proof(induction n)*. Write down the goal of the step case and the induction hypothesis. Explain, pen-and-paper, why the induction hypothesis is not strong enough to prove the step case.

(b) Now prove the lemma, first pen-and-paper and then formally in Isabelle, by generalising the induction hypothesis over m , i.e. with *proof(induction n arbitrary: m)*. Your pen-and-paper proof should state the generalised induction hypothesis and say with which term you instantiate its universally quantified variable in the step case.

Hints: 1. Use *thm Suc.IH* in the step case to compare the induction hypothesis you get with and without *arbitrary: m*.

2. An instance of a universally quantified fact can be obtained with *of*, e.g. *Suc.IH[of Suc m]*.
3. The following lemmas suffice: *add_acc.simps*, *add_0_left*, *add_Suc*, *add_Suc_right*.

(c) Challenge (optional) The following function multiplies using an accumulator. Prove, pen-and-paper and in Isabelle, that it computes $acc + n * m$. Which variable must you generalise over this time, and why?

```
fun mult_acc :: nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat where
  mult_acc 0 m acc = acc
| mult_acc (Suc n) m acc = mult_acc n m (acc + m)
```

```
lemma mult_acc_correct: mult_acc n m acc = acc + n * m
```

Hint: The following lemmas suffice: *mult_acc.simps*, *mult_0*, *add_0_right*, *mult_Suc*, *add.assoc*.